

Beyond Policy-as-Code: Admissibility as a Runtime Primitive for Autonomous Systems

How Constitutional Computing builds on reference monitors, capability security, policy enforcement and least privilege for systems that generate their own execution paths

PUBLISHER

KeyWow

DOMAIN

Execution governance for autonomous AI

OBJECTIVE

Governed Autonomy

– Abstract

Autonomous AI systems change an assumption that has shaped software security for decades. Traditional applications largely execute paths designed in advance by developers, so security mechanisms can focus on controlling identities, permissions, resources, interfaces and predefined operations.

Autonomous systems are different. They can generate intermediate goals, select tools dynamically, compose actions, discover alternative execution paths and propose state changes that were never explicitly encoded as application workflows.

This does not make traditional security obsolete. It makes it more important. Reference monitors, least privilege, capability security, access control, sandboxing, Zero Trust, policy-as-code, isolation and formally verified enforcement remain foundational. But increasingly autonomous systems introduce an additional problem.

How should a system determine whether a specific, dynamically generated consequential action is admissible at the moment it is about to affect the world?

At KeyWow, we refer to the computational treatment of this problem as **Constitutional Computing**. The claim is not that pre-execution authorization is new, nor that existing policy engines cannot evaluate rich contextual information. The claim is narrower: systems that generate their own execution paths benefit from treating action-specific admissibility as an explicit runtime concern, distinct from persistent identity, general permission and raw technical capability.

Crucially, this does not require the reasoning process itself to become deterministic.

THESIS

We do not require deterministic reasoning. We require deterministic consequence boundaries.

This paper defines that boundary, explains its relationship with established security mechanisms, and identifies the engineering properties and tradeoffs required to make the concept operational rather than philosophical.

01 Start with what already exists

Any serious discussion of execution governance has to begin with the foundations already established by computer security.

Least privilege	Limits authority to what a principal requires.
Capability security	Models authority through possession of explicit capabilities.
Reference monitors	Security-sensitive operations should be subject to complete mediation by a tamper-resistant mechanism small enough to analyze.
MAC · DAC · RBAC · ABAC	Regulate operations against resources.
Policy engines	Support contextual and declarative authorization decisions.
Zero Trust	Rejects implicit trust based merely on network location or prior access.
Isolation	Constrains what processes can technically reach.

These mechanisms solve real problems. Constitutional Computing does not replace them. It depends on them. The relevant question is therefore not *how do we invent authorization?*

What additional execution semantics become useful when the system requesting actions can itself generate novel execution paths at runtime?

02 The existing security model

A large class of security decisions can be represented abstractly as a single question: *can principal P perform operation O on resource R under context C?*



Different systems answer it differently. A capability system may focus on possession of authority. An ABAC system may evaluate attributes. A policy engine may evaluate structured context. A reference monitor may ensure that every relevant operation passes through an enforcement boundary.

These mechanisms remain valid for autonomous systems; nothing about AI removes the need for them. The challenge appears when authorization of an operation is no longer equivalent to authorization of the broader consequential action being generated by the system.

03 From operations to consequential actions

Consider an autonomous infrastructure agent. It possesses valid credentials. Its identity is correctly authenticated. Its IAM role permits termination of compute instances. It proposes to terminate 300 production instances because doing so appears to satisfy the current optimization objective.

A conventional authorization system may correctly determine that the principal is permitted to invoke the termination API. But a broader execution question remains.

Should this particular consequential action be allowed to occur under the current conditions?

The same principal can possess the same credentials and invoke the same API while proposing actions with radically different operational consequences. For autonomous systems, the object of governance may therefore need to extend beyond the individual operation.

04 The ingestion dilemma

Autonomous reasoning is often probabilistic, generative and open-ended. Governance needs something structured enough to evaluate. So how does an emergent agent intention become a governable action without introducing either another probabilistic model into the trusted decision path, or a rigid predefined schema for every possible behavior? This is the **ingestion dilemma**.

A weak solution would try to formalize the agent's reasoning itself. That is neither necessary nor desirable. The stronger separation is three-layered.

REASONING SPACE

The agent may reason probabilistically, explore, generate novel plans, and use language, models, tools and heuristics.

CONSEQUENCE BOUNDARY

Before producing a consequential external effect, the intended action has to be expressed through an inspectable execution interface.

EXECUTION SPACE

Only actions that cross that boundary in an admissible form can affect the governed environment.

Open-ended reasoning may remain probabilistic. Consequential execution should not depend on an unstructured internal intention.

The governance boundary does not need to understand the entire chain of thought. It needs to understand enough about the external action being requested to determine whether that action may proceed.

05 Deterministic consequence boundaries

A Constitutional Computing architecture does not attempt to make cognition deterministic. It attempts to make consequential effects pass through a deterministic boundary.

```
Open-ended reasoning
↓ planning and strategy generation
↓ capability selection
↓ consequence boundary
↓ structured execution request
↓ admissibility determination
↓ controlled execution
```

The exact form of the execution request may differ by domain. In a cloud environment it may correspond to a bounded infrastructure action; in finance, to a transaction; in robotics, to an actuator command; in an enterprise workflow, to a state-changing business operation.

The agent may generate arbitrary strategies, but real-world effects must ultimately pass through interfaces whose consequences can be inspected and governed.

The important property is not universal schema design. This is different from requiring every strategy to be anticipated in advance.

06 Capability, permission, authorization and admissibility

These four concepts should remain separate.

Capability

The system is technically able to perform an operation.

Permission

A policy permits a defined class of operation.

Authorization

A principal currently possesses authority to perform that class of operation.

Admissibility

The specific proposed action satisfies the execution conditions that apply to it under the relevant current circumstances.

Capability does not imply authorization, and authorization does not necessarily imply admissibility of every consequential use of that authority. The difference is not simply that admissibility includes more context — ABAC and general policy engines can already consume very rich context. The distinction is architectural.

Admissibility concerns whether a specific consequential action should become execution under the conditions that apply to it now.

07 Why this is not simply "OPA with more fields"

This objection deserves a direct answer. A general policy engine can evaluate actor, action, resource, context, risk, time and environment, and return a decision. Constitutional Computing does not require replacing that evaluator — a policy engine may be part of an implementation. The architectural distinction lies in the role the decision plays.

IN A CONSTITUTIONAL EXECUTION MODEL

01 A consequential action is considered before execution.

02 Its admissibility is determined under current conditions.

03 An affirmative determination is required before execution can proceed.

04 Material changes can invalidate the basis for that determination.

05 Execution must occur through a trusted enforcement path.

06 Successful invocation and verified completion remain different concepts.

The evaluator could be OPA. The architecture is not OPA. OPA can answer policy questions; Constitutional Computing is concerned with making a current admissibility determination a required property of consequential execution.

08 Relationship to the reference monitor

The reference monitor is especially important prior art. Its classic properties — complete mediation, tamper resistance and analyzability — overlap directly with any serious claim of non-bypassable execution governance. Constitutional Computing should therefore be understood as inheriting, not replacing, the reference-monitor principle.

The extension is in the object being mediated. Traditional reference-monitor architectures typically mediate accesses or security-sensitive operations. Autonomous systems increasingly require governance over actions that may be dynamically generated, composed and adapted during execution. The constitutional question becomes: *what consequential effect has actually been admitted to happen?*

If execution can bypass the enforcement boundary, the constitutional model has failed regardless of how sophisticated its decision process may be.

09 Admissibility outcomes

A useful admissibility model should avoid forcing every possible action into a premature binary decision. At a conceptual level, a system may need to distinguish among four outcomes.

Admitted	The relevant execution conditions are satisfied.
Admitted with controls	Execution is permitted only subject to additional explicit constraints.
Blocked	A known applicable condition prohibits execution.
Unresolved	The system cannot currently establish admissibility.

The important point is not the labels themselves; many existing policy systems already distinguish indeterminate outcomes. The relevant execution property is narrower.

An unresolved condition must not silently become permission.

At the same time, unresolved should not necessarily mean permanently prohibited. It should trigger an appropriate resolution path.

10 Preserving autonomy without turning novelty into permission

This creates a legitimate tension. If governance rules are completely rigid, autonomous systems become brittle: they encounter legitimate situations policy designers did not anticipate and stop functioning. If unknown behavior is implicitly allowed, autonomy can become self-authorization. The objective is to preserve two properties simultaneously.

Safety

Known inadmissible actions should not execute.

Liveness

Legitimate novel actions should retain a practical path toward resolution and execution.

That requires distinguishing **prohibited** from **not yet resolvable**. The goal is not to eliminate uncertainty. It is to prevent uncertainty from being silently resolved by the same autonomous component that wants to act.

11 Human review is not a security oracle

Human review is sometimes necessary. But it should not be treated as infallible. Humans introduce latency, inconsistency, fatigue, rubber-stamping, insider risk and incomplete context.

Human escalation is therefore one resolution mechanism among several. Moving a decision to a human does not remove the need for governance: the authority, scope and conditions of human intervention should themselves remain controlled.

12 Composition is a first-class problem

Autonomous systems rarely perform isolated actions. They execute sequences, act in parallel, and multiple agents may coordinate. This introduces a challenge that operation-level authorization alone cannot always capture: an action may be acceptable individually but problematic when combined with another.

Sequential effects

Two individually acceptable actions produce an unsafe result when performed in sequence.

Parallel effects

Two valid actions conflict when executed concurrently.

Accumulative effects

Each operation stays within a local limit, while the aggregate effect becomes unacceptable.

Cross-agent effects

Multiple independently authorized agents collectively produce a state that none should have been able to create alone.

Individual admissibility does not guarantee admissibility of the resulting composition.

This is not uniquely an AI problem. Autonomous systems make it more frequent because execution paths are increasingly assembled at runtime.

13 The execution-neighborhood tradeoff

Composition creates another hard engineering problem: how much surrounding state should governance consider?

TOO NARROW

Cumulative effects may be missed. Cross-agent interactions may remain invisible. Local decisions may create globally unsafe outcomes.

TOO BROAD

Latency increases. Distributed state becomes expensive to synchronize. Consistency becomes harder. Large portions of the system may block each other.

There is no universal answer. A practical architecture needs a bounded **execution neighborhood**: enough context to detect materially relevant interactions without attempting to construct a global model of the environment. Relevant context may

include concurrent activity, shared resources, known dependencies, cumulative limits, protected scopes and explicitly represented relationships.

The important point is that the boundary itself creates a measurable tradeoff — **coverage versus latency, and local autonomy versus systemic awareness** — and that tradeoff should be evaluated empirically rather than hidden behind an assumption of perfect visibility.

14 Time-of-check vs. time-of-use

Admissibility cannot be assumed to last forever. An action may be acceptable when evaluated and no longer acceptable when execution begins. Between evaluation and use, policy may change, authority may be revoked, system state may change, another agent may act, dependencies may shift, or risk conditions may change. This is the classic Time-of-Check / Time-of-Use problem.

An admissibility decision should remain valid only while the material conditions on which it depended remain valid.

If those conditions change sufficiently, execution should require reevaluation. The exact representation and invalidation mechanism are implementation-specific; the requirement is that stale admissibility must not silently remain executable.

15 Non-bypassability and the trusted computing base

"Non-bypassable" is a strong claim. It cannot be guaranteed by application logic alone: a complete deployment depends on a trusted enforcement substrate. Depending on the environment, that may involve workload identity, operating-system controls, network enforcement, container isolation, cloud IAM, gateways, hypervisor or kernel mechanisms, privileged execution services, or hardware-backed trust.

Constitutional Computing can define the requirement — **consequential execution must pass through the trusted enforcement boundary** — but whether that requirement actually holds depends on the system in which the architecture is deployed.

This is an important limitation. If the trusted enforcement substrate is compromised, guarantees above it may no longer hold. Constitutional Computing does not replace low-level security. It depends on it.

16 Threat model

A credible architecture should state what it is trying to defend against.

IN SCOPE

Dynamically generated actions · unexpected planning · legitimate credentials used in inappropriate contexts · prompt injection influencing action selection · excessive standing authority · tool misuse · compromised application-level orchestration · unanticipated composition of actions · stale execution decisions · ambiguous completion claims.

NOT INDEPENDENTLY SOLVED BY THIS LAYER

Compromised kernels or hypervisors · compromise of the trusted enforcement root · malicious root administrators · compromise of all relevant policy authorities · physical compromise · corrupted hardware trust anchors.

No execution-governance layer can compensate for a completely compromised trusted computing base.

17 Trust model

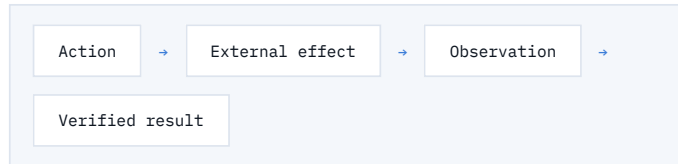
The architecture necessarily depends on trusted components. Depending on the environment, these may include identity providers, policy sources, context providers, enforcement points, trusted time or versioning mechanisms, evidence sources and integrity-protected decision records.

The objective is not to eliminate trust. It is to minimize, isolate and make explicit the components whose integrity the execution guarantee depends on.

This remains consistent with long-standing secure-system design.

18 Execution is not verification

Execution and validation should remain separate concerns. An agent may invoke an operation successfully. A workflow may return HTTP 200. A tool may report completion. None necessarily proves that the intended real-world effect occurred correctly. For consequential actions, a stronger model may be required.



The component that acts should not always be the sole authority determining whether its action succeeded.

The strength of verification should scale with the consequence of being wrong.

19 Verification should be proportional to consequence

Independent verification introduces cost: additional reads, cross-system confirmation, external telemetry, synchronization, waiting for eventual consistency, and human confirmation in exceptional cases. Requiring the strongest possible verification after every low-impact action would make many autonomous systems operationally impractical. The relevant principle is therefore proportionality.

Lower consequence	Execution may proceed with lightweight or asynchronous verification.
Moderate consequence	Execution may proceed with explicit follow-up validation.
High consequence	Downstream execution may depend on stronger verification.
Critical consequence	Verification may become part of the synchronous control path.

The exact classification is domain-dependent. The architectural point is that verification strength should scale with consequence rather than impose a universal synchronous barrier.

20 Provisional and verified state

Execution may produce a state that is operationally **provisional** before it becomes **verified**. For some workflows, *execute* → *continue* → *verify asynchronously* may be acceptable. For others, *execute* → *verify* → *unlock dependent action* may be required.

This prevents closure semantics from becoming a universal latency tax. It also makes the consistency model explicit: the system must know whether a downstream action depends merely on invocation or on verified completion.

21 What counts as independent evidence?

"Different service" does not necessarily mean "independent". Two services may depend on the same compromised source. Evidence independence should therefore be evaluated relative to the trust boundary.

Useful forms of evidence may include confirmation from the destination system, independent transaction records, separate telemetry, independently controlled sensors, external attestation, and evidence from differently trusted systems. The exact strength required depends on consequence; the architecture does not require every action to use the same verification mechanism.

22 Closure as an execution concern

Most authorization models focus on whether an operation may begin. Autonomous execution creates another question: *how do we know the intended consequential action actually reached an acceptable terminal state?*

LOGGING ASKS

What did the system report doing?

CLOSURE ASKS

Has the intended result been established with sufficient evidence?

For some high-consequence actions, the inability to verify the resulting state may itself be relevant before execution is allowed. The architectural principle is therefore that, where consequence requires it, verifiability should be considered part of execution governance rather than an afterthought.

23 Comparison with existing mechanisms

MECHANISM	NATIVE ABSTRACTION	UNIT OF ENFORCEMENT	RELATIONSHIP
Authentication	Identity	Session / principal	Required foundation
RBAC / IAM	Role and permission	Operation on resource	Supplies authority information
ABAC	Attributes and policy	Contextual operation	Can implement parts of admissibility
Capability security	Explicit authority	Resource capability	Strong capability confinement
Reference monitor	Complete mediation	Security-sensitive operation	Foundational enforcement principle
Zero Trust	Contextual access	Access request / session	Complementary trust model
Policy-as-code	Structured policy decision	Arbitrary structured input	Possible decision mechanism
Sandbox / isolation	Reachability	Process / workload	Constrains technical capability
Workflow engine	Defined state progression	Predefined workflow transition	Useful when paths are known
Observability	Events and telemetry	Executed behavior	Supports evidence and investigation
Constitutional Computing	Consequential action under current conditions	Execution decision and lifecycle	Connects admissibility, enforcement and verifiable outcome

24 Safety and liveness

A system that blocks everything is safe in a trivial sense. It is also useless. A production governance layer needs both: known inadmissible execution should not occur (safety), and legitimate execution should remain practically achievable (liveness).

This creates measurable tradeoffs. Excessive conservatism can destroy the economic value of autonomy. Excessive permissiveness can destroy the meaning of governance. A credible architecture needs to optimize both.

25 Performance is part of the security model

Governance that cannot meet operational requirements will eventually be bypassed by operators. Performance is therefore not separate from security. Relevant measures include decision latency, enforcement latency, unresolved rate, policy evaluation cost, execution-neighborhood resolution cost, verification latency, synchronous verification rate and asynchronous verification backlog.

Different classes of action may justify different depths of evaluation. The architecture should not assume every operation requires maximum scrutiny.

26 Open engineering questions

Constitutional Computing should not pretend the difficult implementation questions are already solved universally. Several remain central.

Action formalization	How can diverse autonomous actions cross a deterministic consequence boundary without requiring developers to predefine every possible strategy?
Neighborhood selection	How much surrounding state is sufficient to detect material composition effects without producing unacceptable distributed-systems overhead?
Decision freshness	How should systems determine when a prior admissibility determination has become stale?
Verification depth	When should outcome verification be synchronous, asynchronous, sampled or unnecessary?
Economic viability	At what consequence level does additional governance cost create more value than latency and complexity destroy?

These are engineering questions. They should be answered empirically.

27 What would falsify the approach?

A technical idea should make claims that can fail. A representative hypothesis:

A constitutional execution layer should reduce the execution of contextually inadmissible actions while preserving a useful rate of legitimate autonomous action within defined operational bounds.

USEFUL PUBLIC METRICS

False admission rate	Actions that should not have executed but did.
False blocking rate	Legitimate actions incorrectly prevented.
Unresolved rate	Actions requiring additional resolution.
Bypass rate	Consequential execution reaching the environment without required governance.
Decision latency	Time required to establish execution eligibility.
Staleness failure rate	Execution proceeding on materially outdated decisions.

Composition failure rate	Unsafe aggregate behavior created from individually acceptable actions.
Verification rate	Consequential actions reaching independently established outcomes where required.
Verification overhead	Latency and resource cost introduced by validation.
Task completion rate	Percentage of legitimate autonomous objectives completed successfully under governance.

The last metric is important. A system that minimizes unsafe execution by preventing useful work is not a successful autonomy architecture.

28 A public evaluation scenario

Consider an autonomous cloud optimization agent with legitimate authority to manage compute resources. A public benchmark might include the following scenarios.

A A low-impact change to clearly non-production resources. Execution should normally proceed.
B A large change affecting protected production workloads and active dependencies. Execution should be prevented.
C An unfamiliar workload where relevant context is unavailable. Execution should not silently proceed; additional resolution is required.
D A higher-impact change where additional authority and explicit controls have been established. Execution may proceed subject to those controls.
E An action is approved, but material conditions change before execution. The earlier determination should not automatically remain valid.
F Multiple individually acceptable actions create an unacceptable aggregate effect. The aggregate effect should be constrained where the composition is visible.
G A reversible, low-impact action whose validation source is eventually consistent. Execution may proceed while verification completes asynchronously.
H A high-consequence action whose downstream operations depend on confirmed completion. Dependent execution should remain constrained until verification is available.

These scenarios make the architectural tradeoffs measurable without assuming a universal implementation.

29 What Constitutional Computing does not claim

Constitutional Computing does not claim to replace IAM, capability security, policy-as-code, reference monitors or Zero Trust. It does not eliminate software vulnerabilities, nor guarantee security after compromise of the trusted enforcement substrate. It does not perfectly predict arbitrary future behavior, eliminate human

judgment, eliminate uncertainty, model every possible downstream consequence, formalize an agent's entire reasoning process, or require synchronous verification for every action.

Its narrower objective: to make the admissibility of dynamically proposed consequential actions an explicit, enforceable and verifiable runtime concern.

30 What KeyWow is claiming

At KeyWow, our public thesis is intentionally narrow. We believe increasingly autonomous systems benefit from five architectural properties.

01	Open reasoning, bounded consequence Agents may reason freely, but consequential effects should cross deterministic and inspectable execution boundaries.
02	Action-specific governance Execution decisions should apply to the consequential action being proposed, not merely to persistent identity or standing permission.
03	Current admissibility Execution should depend on conditions that remain materially valid when action occurs.
04	Trusted enforcement Consequential execution should not bypass the governance boundary that determined its admissibility.
05	Proportional verification Where consequence requires it, execution should remain distinct from sufficiently independent verification of the result, with verification strength proportional to operational impact.

These are architectural properties. Their implementation can differ substantially by domain.

31 Relationship to Governed Autonomy

The goal is not to make autonomous systems behave like predefined workflow engines. That would remove much of their value. We want systems capable of discovering new strategies, adapting, composing tools, responding to unfamiliar situations and operating with increasing initiative.

How do we preserve generative autonomy without turning novel reasoning into self-authorized consequence?

We call the desired outcome **Governed Autonomy**. Constitutional Computing is the architectural direction KeyWow is pursuing beneath it.

– Conclusion

Computer security already knows how to authenticate identities, restrict capabilities, enforce policy, mediate access, isolate processes and verify outcomes. Constitutional Computing should build on all of that.

The emerging challenge is not that those mechanisms stopped working. It is that autonomous systems increasingly generate the paths that lead from objective to execution. That changes the object we need to govern. The relevant questions are no longer only *can this principal perform this operation?* or *does policy permit this request?*

- 01 What consequential action is being proposed?
- 02 Can its external effect cross a deterministic governance boundary?
- 03 Is it admissible under the conditions that apply now?
- 04 Is that determination still valid when execution occurs?
- 05 Does interaction with other activity materially change the answer?
- 06 Can execution bypass the enforcement boundary?
- 07 What degree of verification does its consequence justify?
- 08 And can all of this be done without destroying the usefulness and economics of autonomy?

Those are engineering questions. They are measurable. They are falsifiable. And several remain open research and implementation problems.

THE DOMAIN WE CALL CONSTITUTIONAL COMPUTING

The objective is Governed Autonomy: open-ended intelligence with bounded, governed consequence.

– References

01 **James P. Anderson**

Computer Security Technology Planning Study, Volume II.
ESD-TR-73-51, Vol. II. Electronic Systems Division, Air Force Systems Command. October 1972.
seclab.cs.ucdavis.edu/projects/history/papers/ande72.pdf

02 **Jerome H. Saltzer and Michael D. Schroeder**

“The Protection of Information in Computer Systems.”
Proceedings of the IEEE, 63(9), 1278–1308, September 1975.
doi.org/10.1109/PROC.1975.9939

03 **Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly**

Zero Trust Architecture.
NIST Special Publication 800-207. August 2020.
doi.org/10.6028/NIST.SP.800-207

04 **Open Policy Agent contributors**

Open Policy Agent (OPA).
openpolicyagent.org/docs

05 **Open Policy Agent contributors**

How to Deploy OPA.
openpolicyagent.org/docs/deploy

06 **seL4 documentation contributors**

Capabilities.
docs.sel4.systems/Tutorials/capabilities.html

LICENSE

CC BY-NC-ND 4.0

REPOSITORY

github.com/KeyWow-Global/constitutional-computing

© 2026 KeyWow Global. Licensed under CC BY-NC-ND 4.0.